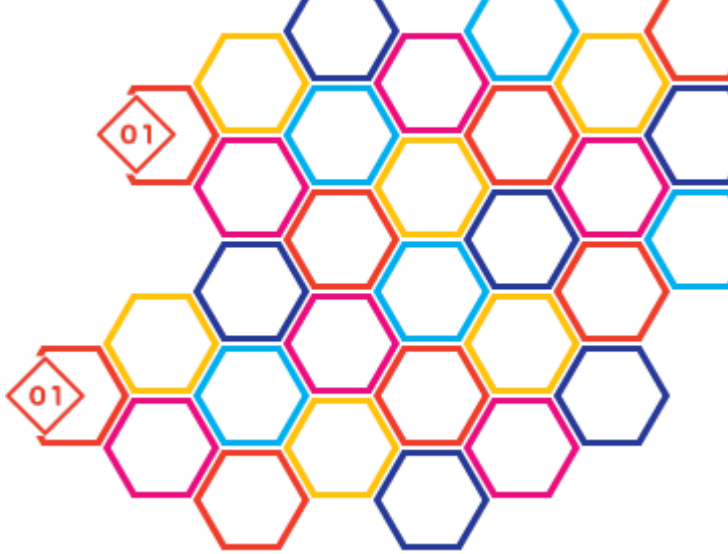


Advanced and Distributed Algorithms
N-ary tree in C++

Pr. Benoît PIRANDA, Ph.D

Member of Computer Science Department of Femto-ST institute

benoit.piranda@femto-st.fr

UNIVERSITÉ MARIE & LOUIS PASTEUR

UFR STGI BELFORT-MONTBÉLIARD

femto-st SCIENCES & TECHNOLOGIES

Outline

- General trees
  - Definition using object
- Traversal
  - Depth-first search
    - Preorder
    - Inorder
    - Postorder
  - Breadth-first search
- Qt application design

General N-ary Tree

- Definition:
  - Attributes
    - Data
    - Children
      - Array of subtrees, pointers (allows direct access to n^{th} child)
      - or list of subtrees, pointers
    - Parent
      - Pointer, not mandatory
      - Optimize some algorithms, allows to identify the root.
  - Methods
    - Constructor
    - Destructor
      - Clear all the memory
    - Add child
    - Print methods

Definition of the class

```
template<class T> class NaryTree {
public:
        NaryTree(T *p_data,NaryTree *p_parent):data(p_data),parent(p_parent) {};
        ~NaryTree() {
                for (auto &child : children) delete child;
                delete data;
        }
        NaryTree<T>*addChild(T *newData) {
                NaryTree<T>* newChild = new NaryTree<T>(newData,this);
                children.push_back(newChild);
                return newChild;
        }
private:
        T *data;
        QVector<NaryTree*> children;
        NaryTree *parent;
};
```

A basic print method

```
std::string print(){
    std::stringstream str;
    str << *data;
    for (auto &child:children) {
        str << child->print();
    }
    return str.str();
}
```

Operator << must be defined for MyData

```
std::ostream& operator<<(std::ostream& out, const MyData &data)
```

```
class MyData {
public :
    int key;
    QString name;
    MyData(int p_key,const QString &p_name):key(p_key),name(p_name) {};
};
std::ostream& operator<<(std::ostream& out, const MyData &data){
    out << "(" << data.key << "," << data.name.toStdString() << ")" << std::endl;
    return out;
}
```

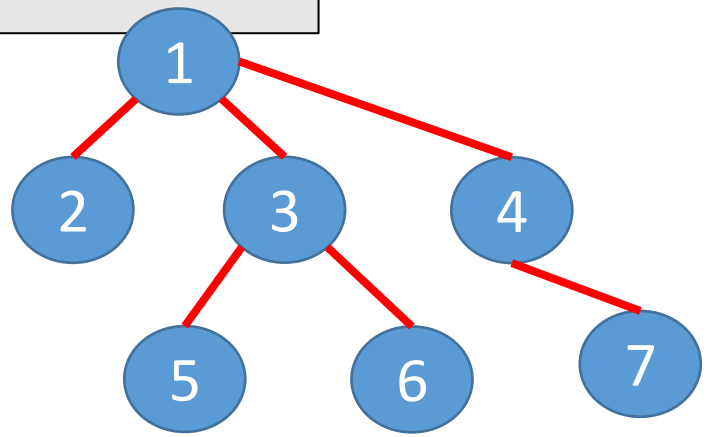
Example of use

```
NaryTree<MyData> *myRoot=new NaryTree<MyData>(new MyData(1,"Alpha"),nullptr);
myRoot->addChild(new MyData(2,"Bravo"));
NaryTree<MyData> *charlie=myRoot->addChild(new MyData(3,"Charlie"));
NaryTree<MyData> *delta=myRoot->addChild(new MyData(4,"Delta"));
charlie->addChild(new MyData(5,"Echo"));
charlie->addChild(new MyData(6,"Foxtrot"));
delta->addChild(new MyData(7,"Golf"));

std::cout << myRoot->print() << std::endl;
```

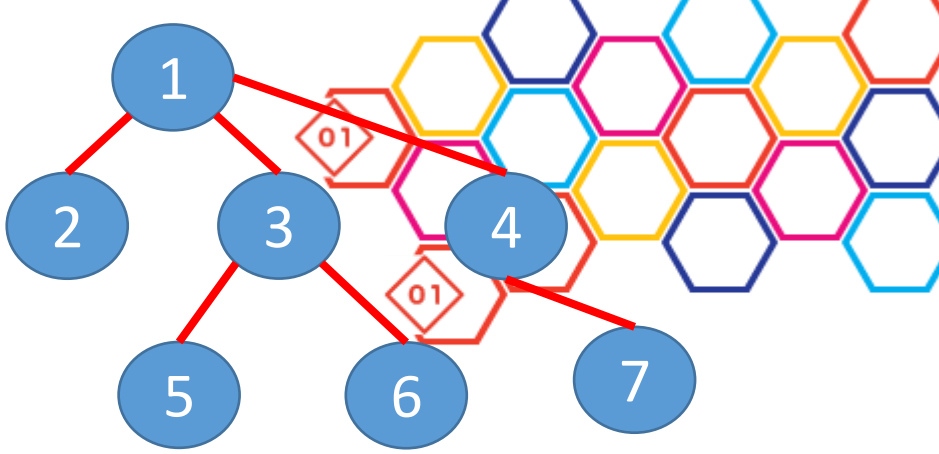
Preorder crossing

(1,Alpha)
(2,Bravo)
(3,Charlie)
(5,Echo)
(6,Foxtrot)
(4,Delta)
(7,Golf)



Basic print method

```
std::string printPostorder(){
        std::stringstream str;
        for (auto &child:children) {
                str << child->printPostorder();
        }
        str << *data;
        return str.str();
}
```



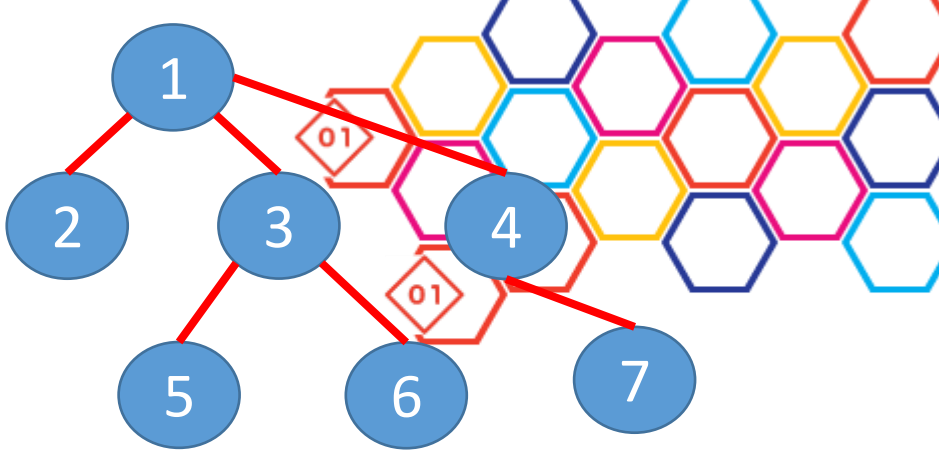
Postorder crossing

(2,Bravo)
(5,Echo)
(6,Foxtrot)
(3,Charlie)
(7,Golf)
(4,Delta)
(1,Alpha)

Basic print method

```
std::string printInorder(){
        std::stringstream str;
        auto child=children.begin();
        if (child!=children.end()) {
                str << (*child)->printInorder(); child++;
        }
        str << *data;
        while (child!=children.end()) {
                str << (*child)->printInorder();
                child++;
        }
        return str.str();
}
```

Inorder crossing



(2,Bravo)
(1,Alpha)
(5,Echo)
(3,Charlie)
(6,Foxtrot)
(7,Golf)
(4,Delta)

Breadth First Search (BFS)

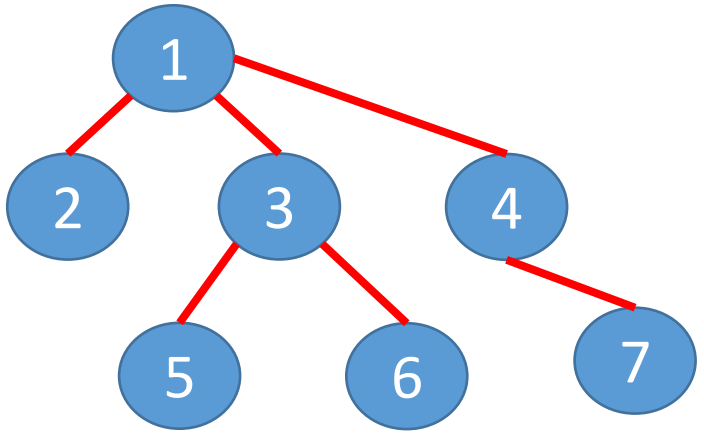
- Print all the children with the same depth:
  - First the root,
  - Then, all the children of the root (stored in children list)
  - All the children of all the nodes of the previous level:
    - We need a structure to store all the children of the same generation.
    - A list a pointer to nodes (QList<NaryTree*>)
- Only one list is necessary:
  - Add new nodes at the end,
  - Consume nodes from the beginning.

Breadth First Search (BFS)

```
std::string printBreadthFirst() {
        QList<NaryTree*> current;
        std::stringstream str;
        current.push_back(this);
        while (!current.isEmpty()) {
                auto node=current.front();
                current.pop_front();
                str << *(node->data);
                // add root children in the list
                for (auto &child:node->children) {
                        current.push_back(child);
                }
        }
        return str.str();
}
```

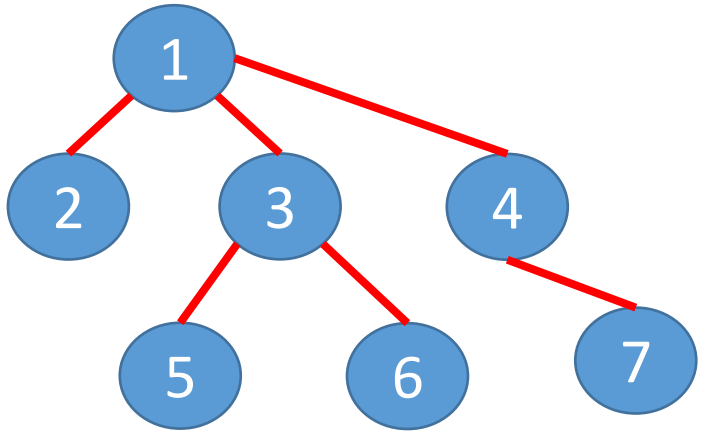
Use a list to store the nodes of same depth:

QList<NaryTree*> current;



(1,Alpha)
(2,Bravo)
(3,Charlie)
(4,Delta)
(5,Echo)
(6,Foxtrot)
(7,Golf)

```
std::string printBreadthFirst() {
        QList<NaryTree*> current;
        std::stringstream str;
        current.push_back(this);
        int n=1;
        while (!current.isEmpty()) {
                auto node=current.front();
                current.pop_front();
                n--;
                str << *(node->data);
                // add root children in the list
                for (auto &child:node->children) {
                        current.push_back(child);
                }
                if (n==0) {
                        str << "----------" << std::endl;
                        n=current.size();
                }
        }
        return str.str();
}
```



(1,Alpha)

(2,Bravo)
(3,Charlie)
(4,Delta)

(5,Echo)
(6,Foxtrot)
(7,Golf)

Develop using Qt Creator

- What is Qt?
  - A cross-platform application development framework for desktop, embedded and mobile.
  - For Linux, OS X, Windows, Android, iOS, BlackBerry and others.
  - Qt is a framework written in C++.
  - It can be compiled by any standard compliant C++ compiler like Clang, GCC, ICC, MinGW and MSVC.
  - A preprocessor, the MOC (Meta-Object Compiler), is used to extend the C++ language with features like signals and slots. Before the compilation step, the MOC parses the source files written in Qt-extended C++ and generates standard compliant C++ sources from them.
- Qt Creator is a cross-platform (Windows, Linux, Mac) IDE
  - Part of the Qt SDK,
  - Simplify the development of cross-platform GUI applications.

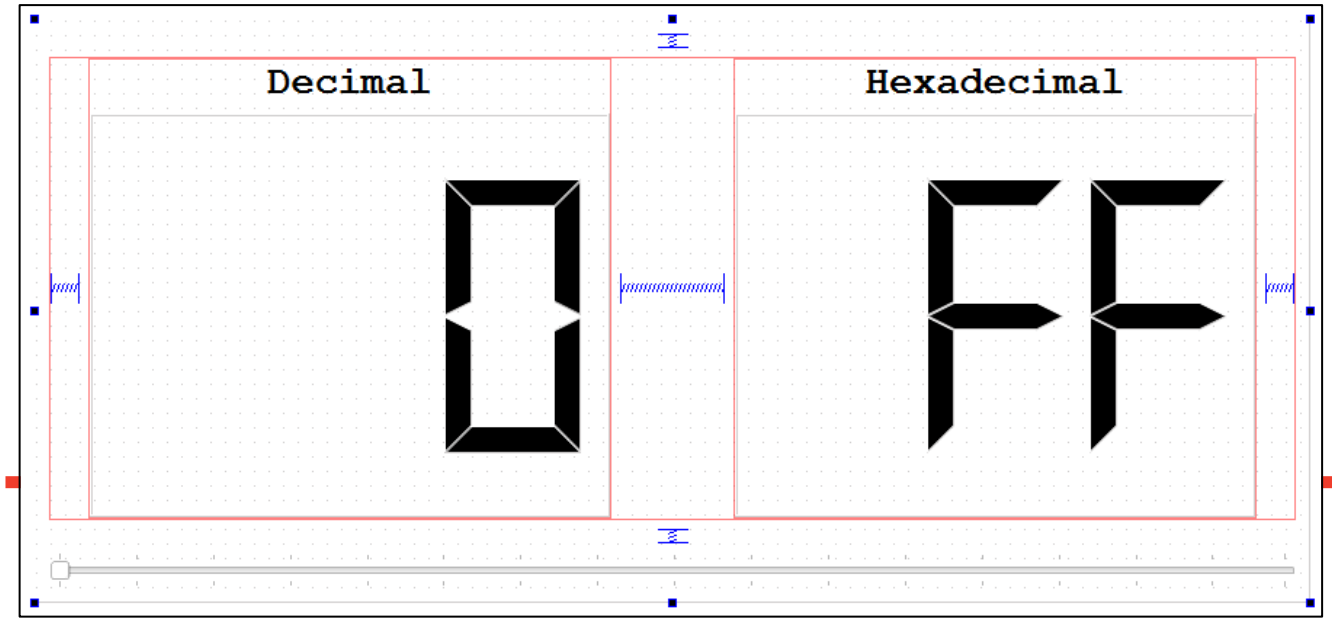
Develop using Qt Creator

- Step by step:
  - Create a project
  - Draw the interface
    - Widgets
    - Menu
    - …
  - Add interactions between widgets

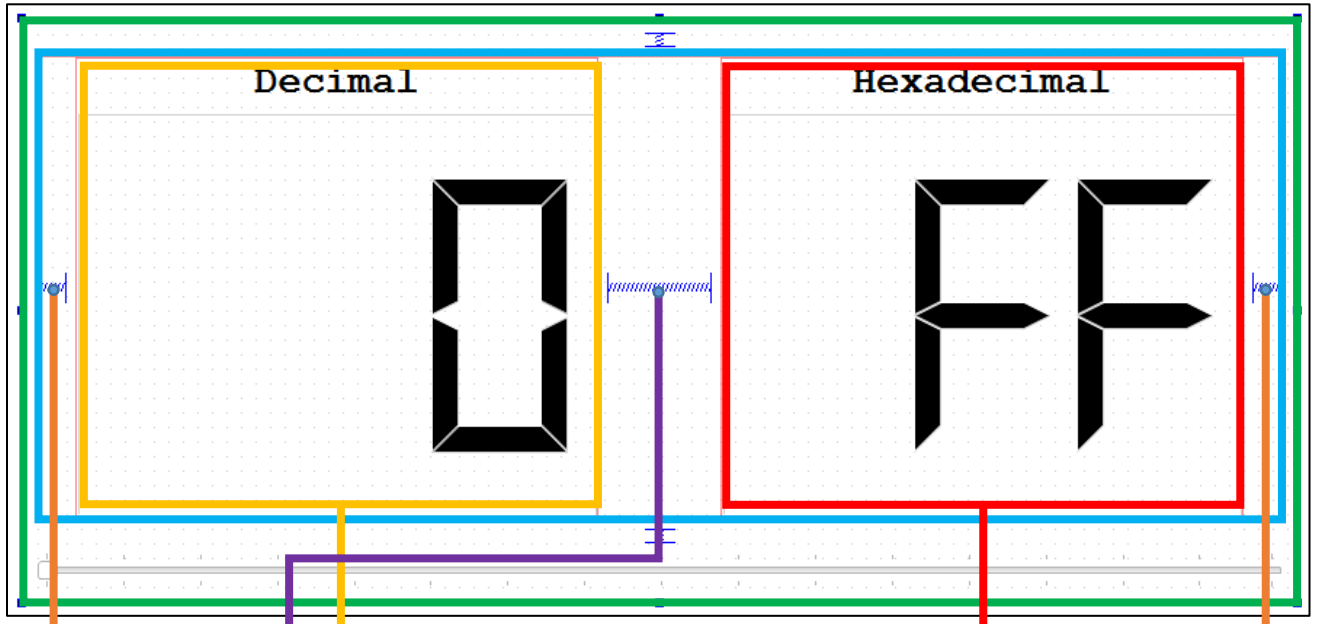
Creating an basic interface

- Click in **`forms/widget.ui`**
- Define the hierarchy of the interface
  - Widget is a set as **`verticalLayout`**
    - All chidren are aligned one below the other.
  - Distribution of **`horizontalLayout stretch`** is [0,5,1,5,0]

Objet	Classe
Widget	QWidget
horizontalLayout	QHBoxLayout
VL_Decimal	QVBoxLayout
Decimal	QLabel
myLCD	QLCDNumber
VL_Hexa	QVBoxLayout
label	QLabel
myHexaLCD	QLCDNumber
spacerCenter	Spacer
spacerLeft	Spacer
spacerRight	Spacer
horizontalSlider	QSlider
spaceMiddle	Spacer
spaceTop	Spacer



Creating a basic interface



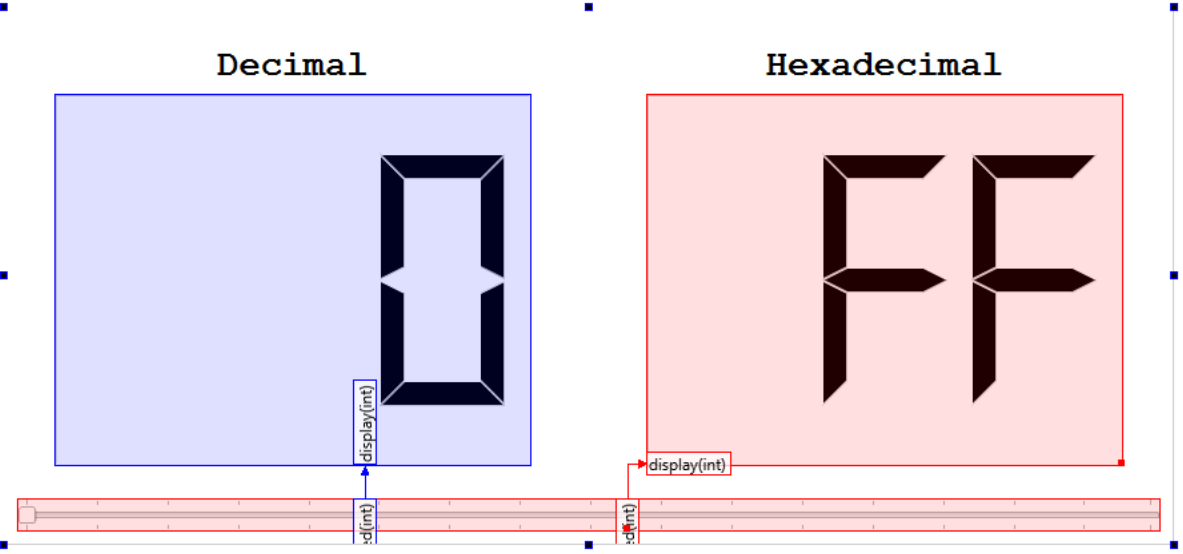
Stretch :

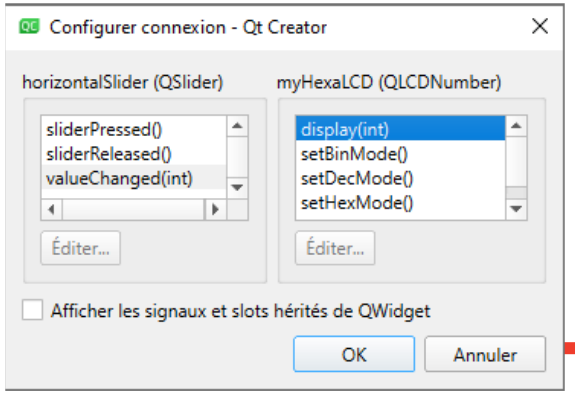
- 0 = use widget size
- 5 = 5/11 of the width
- 1 = 1/11 of the width

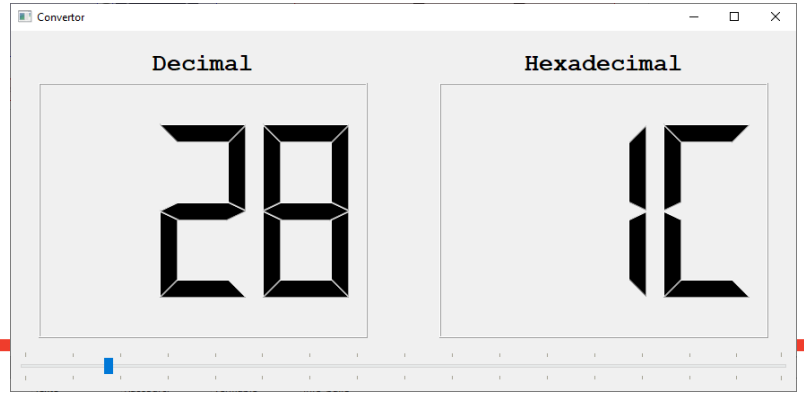
Objet	Classe
Widget	QWidget
horizontalLayout	QHBoxLayout
VL_Decimal	QVBoxLayout
Decimal	QLabel
myLCD	QLCDNumber
VL_Hexa	QVBoxLayout
label	QLabel
myHexaLCD	QLCDNumber
spacerCenter	Spacer
spacerLeft	Spacer
spacerRight	Spacer
horizontalSlider	QSlider
spaceMiddle	Spacer
spaceTop	Spacer

Qt Tips: Slot and Signals

- It is possible to create a connection between two widgets directly from the interface:
  - An action on one widget produces an event in another one.
    - Drag drop from the signal to the slot
  - To change the value of a LCD display from a slider:
    The "**`value change(int)`**" signal of the **`horizontalSlider`** must be connected to the **`myHexaLCD`** "**`display(int)`**" slot.







Creating your own widget

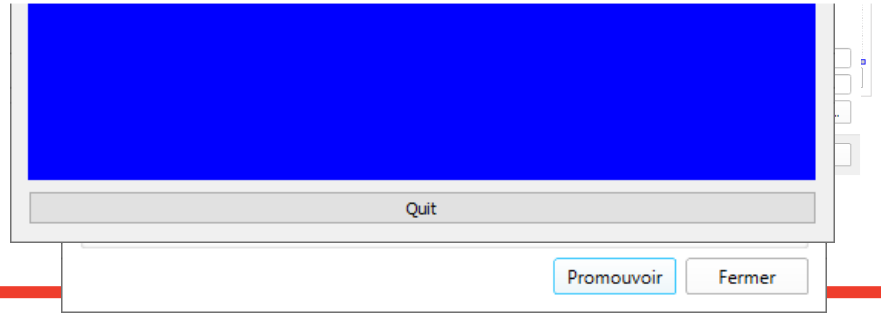
- Create a basic application and add an empty « widget » in your interface
- Then create a class that inherit from the QWidget class
  - **`File/New File`**…
  - Here class name is myWidget
- Add the paintEvent method
  - void ***paintEvent***(QPaintEvent*) override;
  - Just fill your drawing area in blue to test.
- In the designer interface
  - Right click on the widget and select « promote to »
  - Promote to **`myWidget`** class

```
#include "mywidget.h"
#include <QPainter>

myWidget::myWidget(QWidget *parent)
    : QWidget{parent}
{

}

void myWidget::paintEvent(QPaintEvent*) {
    QPainter painter(this);
    painter.fillRect(0, 0, width()-1, height()-1, Qt::blue);
}
```

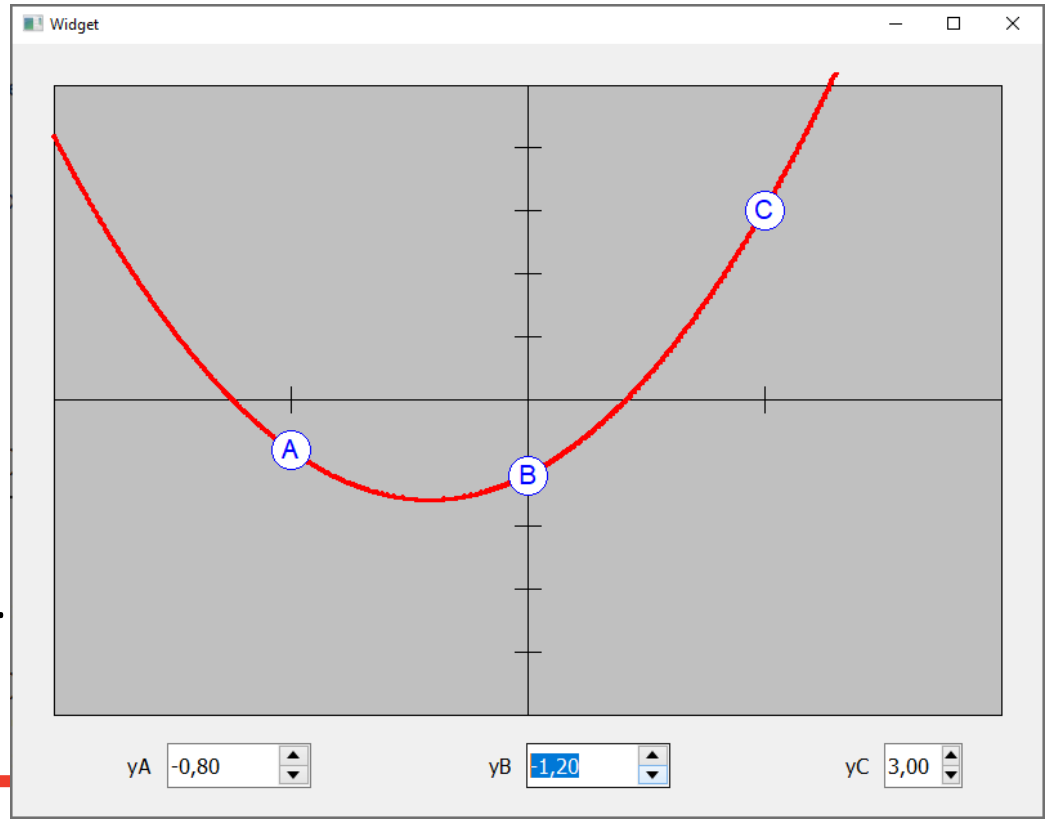


Manage events

- Override other methods
  - Window events
    - void ***resizeEvent***(QResizeEvent ***event**) override;
    - void ***showEvent***(QShowEvent*) override;
  - Mouse events
    - void ***mousePressEvent***(QMouseEvent*) override;
    - void ***mouseMoveEvent***(QMouseEvent*) override;
    - void ***mouseReleaseEvent***(QMouseEvent*) override;
  - Keyboard events
    - void ***keyPressEvent***(QKeyEvent*) override;
    - void ***keyReleaseEvent***(QKeyEvent*) override;

Signal and Slots in your widget

- Exercise:
  - Create a drawing area that draws a quadratic curve passing through the three following points:
    - **`A(-1,yA)`**
    - **`B(0,yB)`**
    - **`C(+1,yC)`**
  - Connect **`SpinBoxes`** to the **`Drawing area`**
    - Updating values in **`yA`**, **`yB`** and **`yC`** must update the curve.
  - Connect the **`Drawing area`** to **`SpinBoxes`**
    - Drag-drop points must change **`SpinBoxes`** values.



Signal and Slots in your widget

- Create slots in your class:

```
public slots:
        void setYA(double value) { yA=value; computeCurve(); };
        void setYB(double value) { yB=value; computeCurve(); };
        void setYC(double value) { yC=value; computeCurve(); };
```

- Connect slots and signals:

```
QObject::connect(ui->SByA, SIGNAL(valueChanged(double)),
                 ui->myDrawing, SLOT(setYA(double)));
QObject::connect(ui->SByB, SIGNAL(valueChanged(double)),
                 ui->myDrawing, SLOT(setYB(double)));
QObject::connect(ui->SByC, SIGNAL(valueChanged(double)),
                 ui->myDrawing, SLOT(setYC(double)));
```

Signal and Slots in your widget

- Create signal in your class:

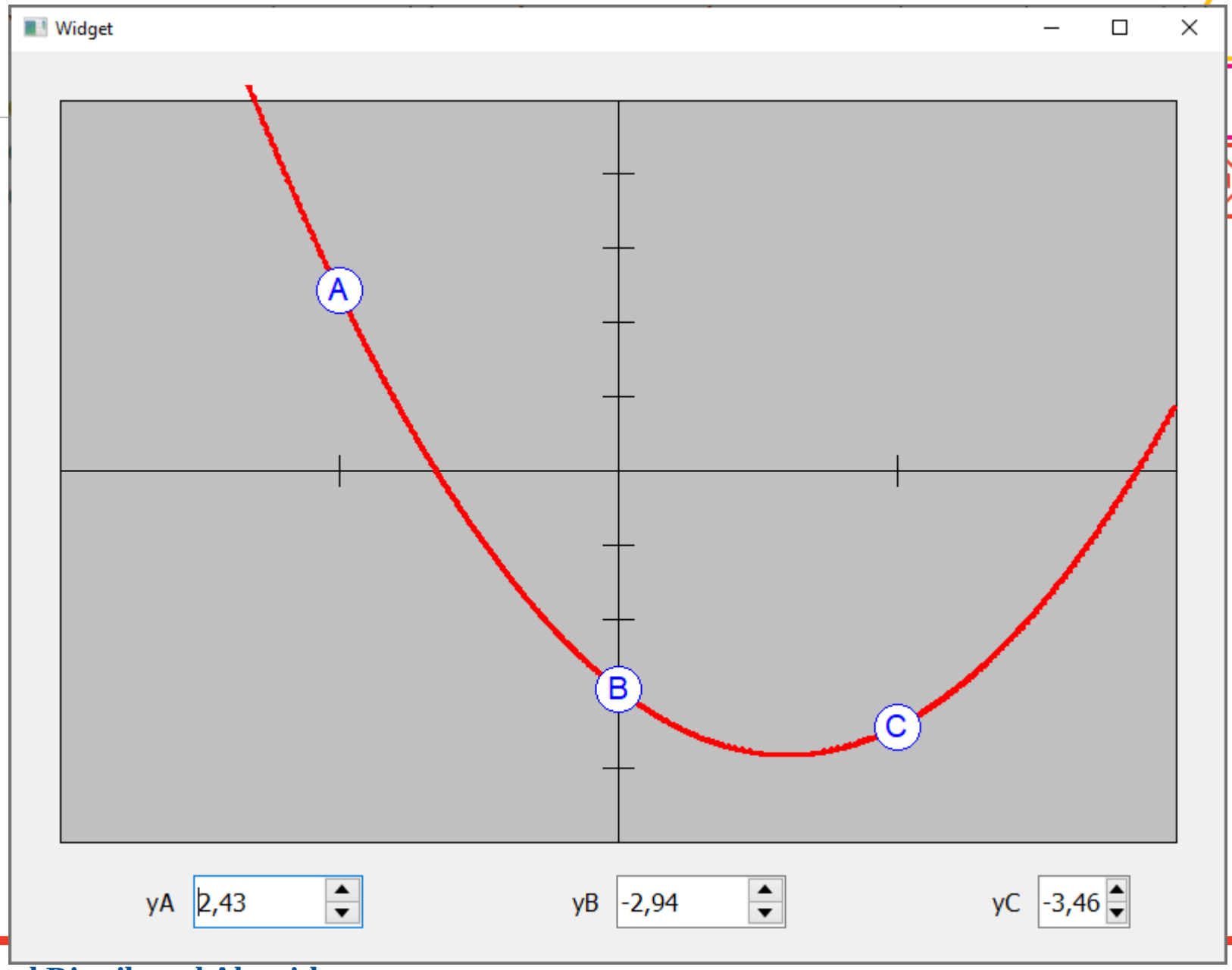
```
signals:
        void YAChanged(double newValue);
        void YBChanged(double newValue);
        void YCChanged(double newValue);
```

- Connect signals and slots

```
QObject::connect(ui->myDrawing, SIGNAL(YAChanged(double)), ui->SByA, SLOT(setValue(double)));
QObject::connect(ui->myDrawing, SIGNAL(YBChanged(double)), ui->SByB, SLOT(setValue(double)));
QObject::connect(ui->myDrawing, SIGNAL(YCChanged(double)), ui->SByC, SLOT(setValue(double)));
```

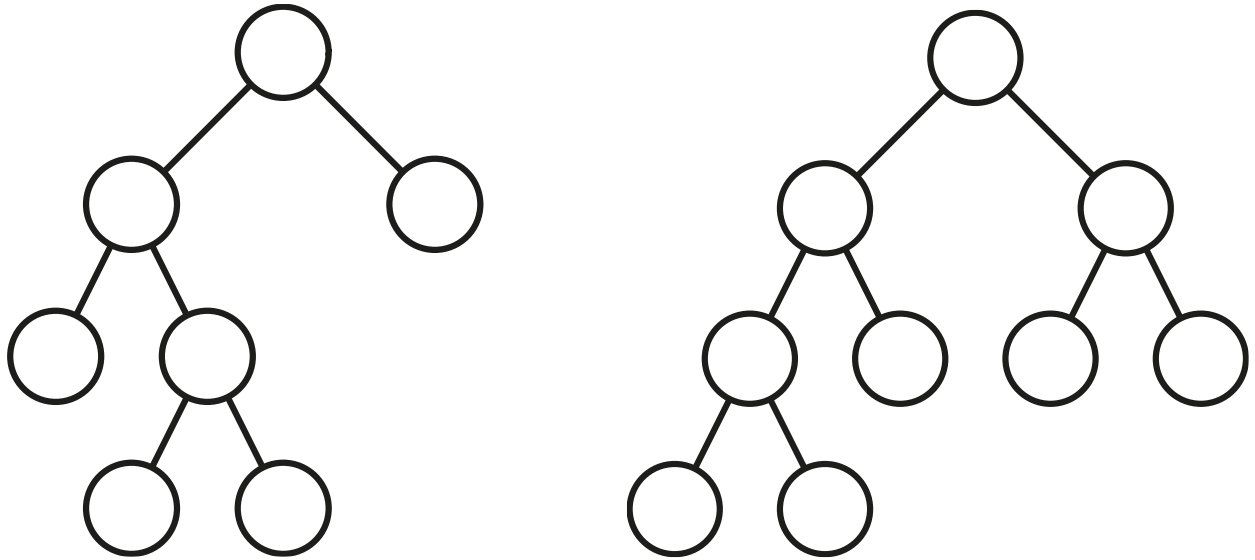
- Emit a signal

```
emit YAChanged(yA);
```


Widget
A
B
C
yA
2,43
yB
-2,94
yC
-3,46

Definitions on Binary Trees

- A **full binary tree** is a binary tree in which each node has exactly zero or two children
- A **complete binary tree** is a binary tree, which is completely filled, with the possible exception of the bottom level, which is filled from left to right

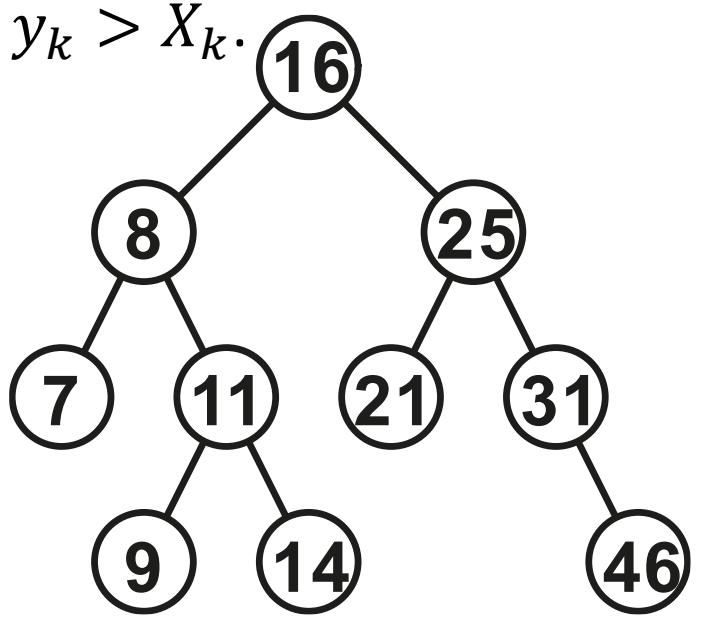


Binary Search Tree

- A Binary Search Tree (BST) contains in each node:
  - A key k, with comparison propriety
    - Considering two nodes X and Y, it exists a relation to compare the keys $X_k < Y_k$.
  - Extra data (optional)
  - Links to left child, right child and parent node.
- Leaf nodes are connected to NIL, (nullptr pointer)
- Only the root node has NIL as parent link.
- Each node of Binary Search Tree verifies the following properties:
  - 1. If Y is in the left sub-tree of the node X then $\forall y \in Y, y_k < X_k$.
  - 2. If Y is in the right sub-tree of the node X then $\forall y \in Y, y_k > X_k$.

Main property of BST

- It is possible to find the path from the root to a node knowing its key!
- Each node of Binary Search Tree verifies the following properties:
  - 1. If Y is in the left sub-tree of the node X then $\forall y \in Y, y_k < X_k$.
  - 2. If Y is in the right sub-tree of the node X then $\forall y \in Y, y_k > X_k$.



Recursive code

- Count the number of nodes
  - Return the sum of the number of nodes of children +1
- Compute the height of the tree
  - Return the max height of the children +1

```
int getHeight() {
    return 1+ qMax((leftChild?leftChild->getHeight():0),
           (rightChild?rightChild->getHeight():0));
}
```

Recursive code

- Check if a tree is full ?
  - If there is exactly one child in the current node -> return false;
  - If 0 or 2, return (leftChild is full?) **and (**right child is full?)

```
bool BST::isFull() {
    // is there exactly One child -> return false
    if ((leftChild == nullptr && rightChild!=nullptr)||
        (leftChild != nullptr && rightChild==nullptr)) return false;
    // left branch is Full is empty or Full
    bool leftFull= (leftChild==nullptr || leftChild->isFull());
    // right branch is Full is empty or Full
    bool rightFull= (rightChild==nullptr || rightChild->isFull());
    // subTree is full if the two branches are full
    return leftFull && rightFull;
}
```

Recursive code

- Check if a tree is complete ?
- The decision can be donne in the penultimate level
  - Consider a variable ***end*** that stores if the left part is full (initially set to false)
  - If ***end*** the node must be a leaf (until the most right node)
  - Else
    - ***end*** became true if (leftChild==null || rightChild==null)
    - The tree **is complete** if (leftChild==null && rightChild==null)

```
bool BST::isComplete(int h,bool &end) {
    if (h==1) { // penultimate level
        if (end) {
            return isLeaf();
        } else {
            if (leftChild==nullptr) {
                end=true;
                return rightChild==nullptr; // true if no right child
            }
            if (rightChild==nullptr) {
                end=true;
                return true;
            }
            return true;
        }
    } else {
        if (leftChild && rightChild) return leftChild->isComplete(h-1,end) && rightChild->isComplete(h-1,end);
        return false;
    }
}
```

Recursive code

- Search if a key ***k*** is in the tree
  - If (***k==node.key***) -> found (return true)
  - Else if (***k<node.key) return (search in left child if exists)***
  - Else ***return (search in right child if exists)***

```
bool BST::findKey(int k) {
    if (key==k) {
        return true;
    } else {
        if (key>k) return (leftChild?leftChild->findKey(k):false);
        else return (rightChild?rightChild->findKey(k):false);
    }
}
```